

TurnosRed

Informe técnico - Actividad 1

Asignatura: Integraciones Web

Fecha de ejecución: 3 de septiembre de 2026

Backend profesional para normalizar, administrar y comunicar en tiempo real turnos médicos provenientes de fuentes JSON heterogéneas.

1. Solución implementada

El proyecto utiliza Node.js 24 LTS, TypeScript en modo estricto, Express y Socket.IO. La arquitectura separa modelos, servicios, controladores, rutas y eventos. La lectura inicial usa **node:fs/promises**, `async/await` y `try/catch`.

La normalización convierte el ID a entero, el documento a texto, elimina espacios redundantes del paciente, transforma fechas DD/MM/AAAA a AAAA-MM-DD, cambia horas con punto a HH:MM y convierte variantes de sí/no en booleanos. Los registros sin la estructura mínima son descartados.

Evidencia de depuración y carga inicial

```
Punto de observación: TurnosService.cargar()  
Carga inicial: 2 aceptados, 1 rechazados.  
TurnosRed disponible en http://localhost:3000  
Verificación TypeScript strict: APROBADA  
Verificación ESLint: APROBADA
```

El archivo `.vscode/launch.json` permite repetir esta sesión con la configuración “Depurar TurnosRed”.

2. Pruebas de la API REST

Los cinco endpoints se ejecutaron contra el servidor local. La colección importable se encuentra en postman/TurnosRed.postman_collection.json y los resultados completos en evidencias/resultados-api.json.

Prueba	Método	Estado	Tiempo
Obtener todos	GET	200	5 ms
Obtener por ID	GET	200	3 ms
Crear turno	POST	201	32 ms
Actualizar turno	PUT	200	14 ms
Eliminar turno	DELETE	200	3 ms

Resultado general: APROBADO. GET devolvió 200, POST devolvió 201, PUT y DELETE devolvieron 200. También se comprobó previamente el 404 para un recurso inexistente.

Ejemplo de respuesta normalizada

```
{
  "id": 102,
  "paciente": "Carlos Ruiz",
  "documento": "31654210",
  "especialidad": "Pediatría",
  "fecha": "2026-08-14",
  "hora": "10:00",
  "confirmado": true
}
```

3. Eventos y comunicación en tiempo real

Cada alta, modificación o baja exitosa emite primero un evento interno desacoplado mediante EventEmitter. El servidor Socket.IO escucha esos eventos y los retransmite a los clientes conectados.

Operación	Evento interno	Evento Socket.IO
POST	turno:creado	turno:nuevo
PUT	turno:actualizado	turno:actualizado
DELETE	turno:eliminado	turno:eliminado

El archivo evidencias/cliente-socket.html funciona como cliente de demostración sin polling. Al abrirlo mientras el servidor está activo, muestra la conexión y cada evento recibido sin recargar la página.

4. Reproducibilidad y entrega

El repositorio local contiene cuatro commits claros: configuración, implementación del backend, documentación y evidencias. El README explica instalación, variables de entorno, scripts, API, eventos y estructura de carpetas.

```
nvm use
npm install
copy .env.example .env
npm run lint
npm run build
npm start

Importar en Postman:
postman/TurnosRed.postman_collection.json

Cliente en tiempo real:
evidencias/cliente-socket.html
```

Conclusión. La solución satisface los requisitos técnicos: tipado estricto, normalización y validación, CRUD con códigos HTTP, persistencia, eventos internos, Socket.IO, herramientas de calidad y documentación.